

CodeGraphNet: Embedding-Driven Enhanced Vulnerability Detection with Line-Level Error Identification

1. Statistics of Datasets

Key lexical and structural features of the gathered vulnerable code samples (from Table 1) are shown by dataset statistics and depicted in Figure 1. Code size varies widely among vulnerability categories, according to the violin plot of function length distributions, with functions associated with CWE-469 and CWE-476 exhibiting relatively higher median lengths and wider spread, suggesting greater code complexity in these vulnerability types.

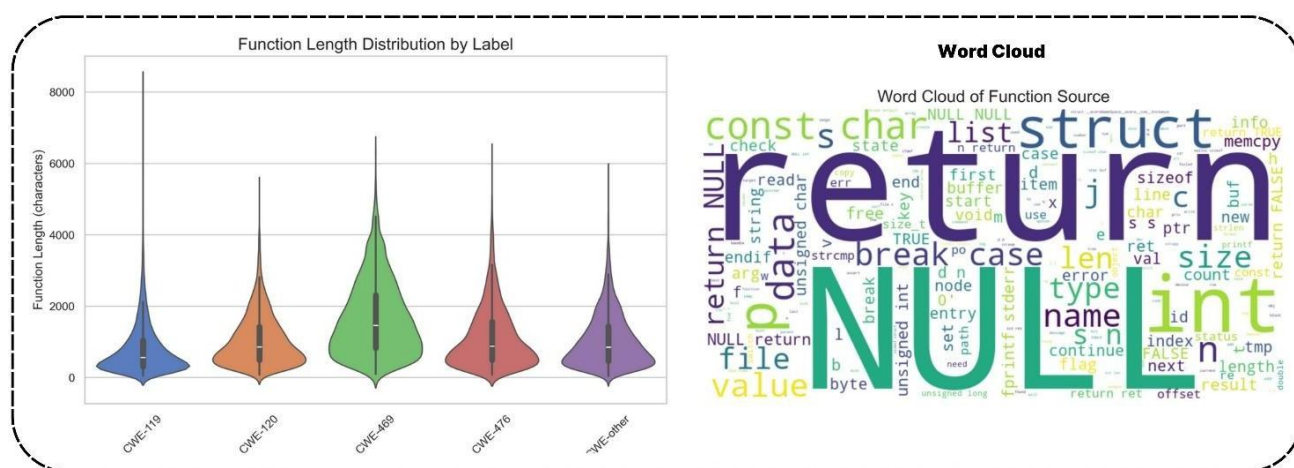


Figure 1: Statistical Overview of our Analysis Dataset.

In contrast, functions in CWE-119 and CWE-120 display more compact distributions, indicating concise code segments where buffer-related flaws are more localized. On the other hand, the word cloud analysis highlights dominant lexical tokens such as *return*, *NULL*, *int*, *size*, *data*, and *char*, reflecting the syntactic and semantic patterns common in low-level C/C++ functions. The predominance of memory and data-handling actions, which are fundamental characteristics of software vulnerabilities recorded in the dataset, is further supported by the frequent occurrence of control-flow keywords and pointer-related terms.

Table 1: Vulnerability Classes and Data Distribution Before and After Balancing.

Class	Before Balancing	After Balancing		
	Number of Data	Number of Data	Training Data	Testing Data
Non-Vul	15,050	4,700	3,800	900
CWE-119	1,21,618	4,502	3,605	897
CWE-120	18,750	4,496	3,557	939
CWE-469	1,008	4,500	3,608	892
CWE-476	8,941	4,503	3,632	871
CWE-other	18,117	4,508	3,605	903
Total Data	1,69,939	27,209	21,807	5,402

2. Model Discussion

In this section we describe the overall model, where Fig. 1 Describe all models basic workflow.

Gated Recurrent Unit (GRU): The Gated Recurrent Unit (GRU) [1] is a form of recurrent neural network (RNN) that is intended to handle sequence data effectively. It has two basic gates, the *reset gate* and the *update gate*, that control the flow of data across the network. The reset gate governs how much prior information to forget, whereas the update gate specifies how much fresh information to store. Unlike standard RNNs, GRUs may successfully capture long-term dependencies by regulating memory using these gates. This makes GRUs appropriate for applications such as time series prediction and natural language processing.

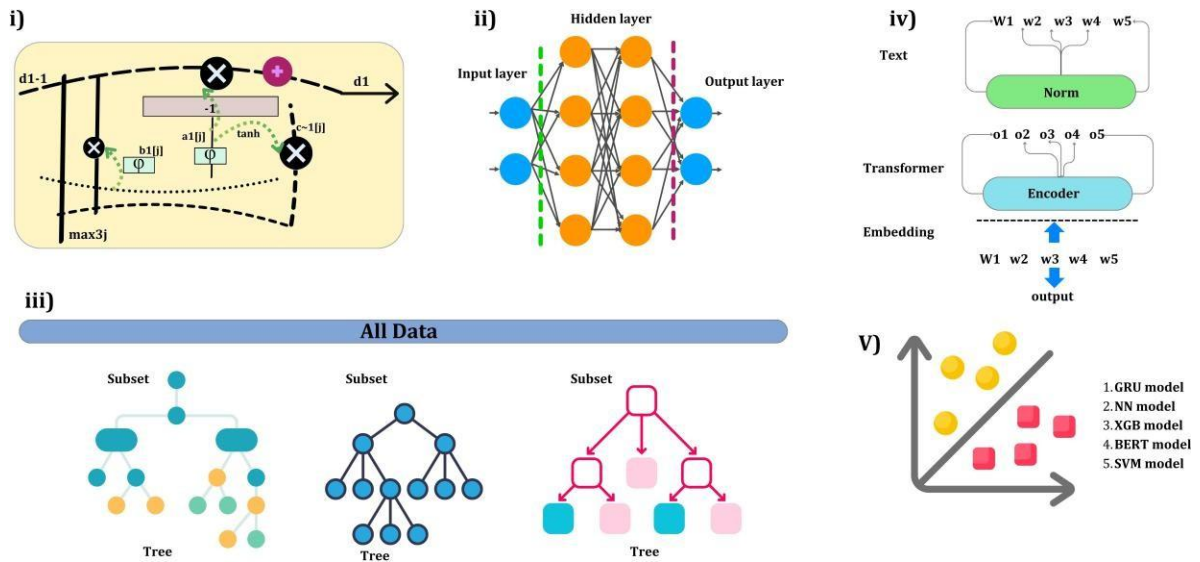


Figure 2: Illustration of the overall models workflow.

Neural Networks (NN): Neural Networks [2] are computer models inspired by the human brain that are used to identify patterns and make predictions. They are made up of layers of linked nodes known as neurons that analyze incoming data and transmit information via weighted connections. Each neuron uses an activation function to decide whether to "fire" and contribute to the next layer. Neural networks generally consist of an input layer, hidden layers, and an output layer.

Ensemble Classifier Model (DeepTree)

We developed an ensemble model that combines deep learning and traditional machine learning models to precisely detect multi-class vulnerabilities in source code termed as DeepTree and the architecture shown in Figure 3. Initially, we trained a DT model on our CodeGraphNet generated dataset, which served to predict class probabilities and the predicted probabilities were subsequently utilized as transformed features for a neural network model. The model is compiled and trained using the Adam optimizer and sparse categorical cross-entropy loss function on the transformed dataset.

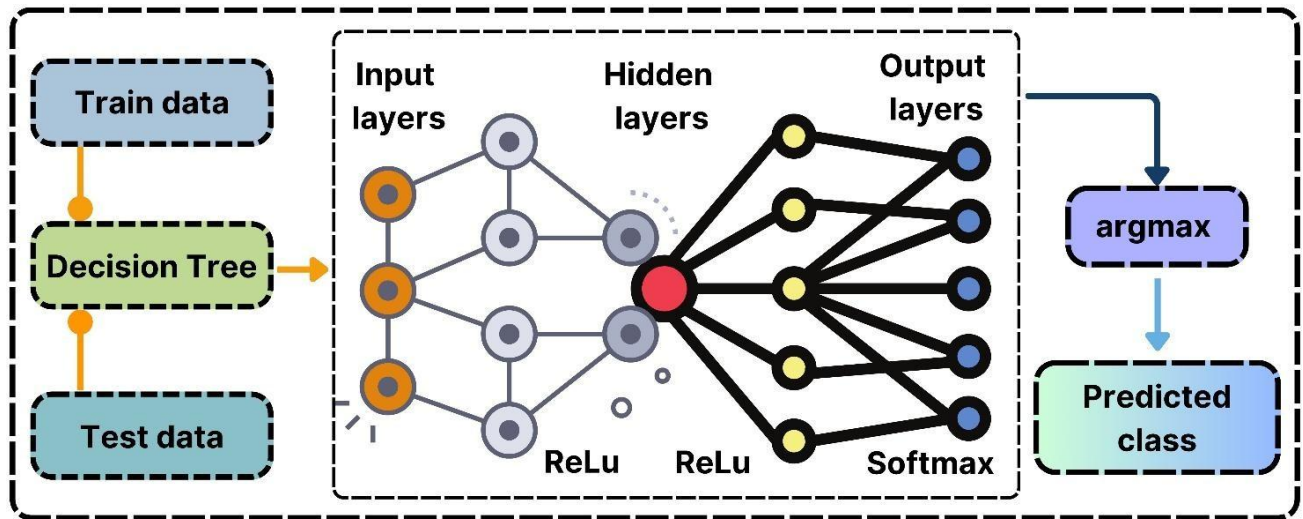


Figure 3: An Insightful Overview of DeepTree's Architectural Design.

Support Vector Machines (SVM): Support Vector Machines (SVMs) [3] are supervised machine learning algorithms that are generally used for classification. SVMs function by determining the best hyperplane for optimally separating data points from distinct classes. This hyperplane is defined by support vectors, which are the points in each class that are closest to the border.

Extreme Gradient Boosting (XGB): XGB [4] is a powerful and effective gradient-boosting machine learning technique that has been improved for speed and performance. XGB constructs an ensemble of decision trees progressively, with each tree correcting the flaws of the preceding ones by minimizing a loss function. It employs regularization to prevent overfitting, hence improving the model's accuracy and resilience, particularly on complex datasets.

LLM Model BERT: BERT [5] is a cutting-edge natural language processing approach created by Google. BERT, unlike standard models that read text sequentially, takes a bidirectional approach, processing text from left to right and right to left. This enables BERT to record the

whole context of a word based on its surrounding words in a phrase, improving its capacity to recognize subtlety and meaning. BERT is pretrained on a large corpus utilizing tasks such as masked language modeling and next sentence prediction.

3. CodeBERT Embedding:

CodeBERT is a large bimodal pre-trained language model which was designed by Microsoft Research with the express purpose of working with programming languages. It was developed using a large collection of source code from several programming languages based on the BERT architecture [6]. CodeBERT efficiently captures the syntactic structure and semantic meaning of code fragments by producing contextual embeddings for them. The architecture is depicted in Figure 3. These embeddings are useful for a variety of software engineering activities, such as vulnerability identification, code completion, and summarization.

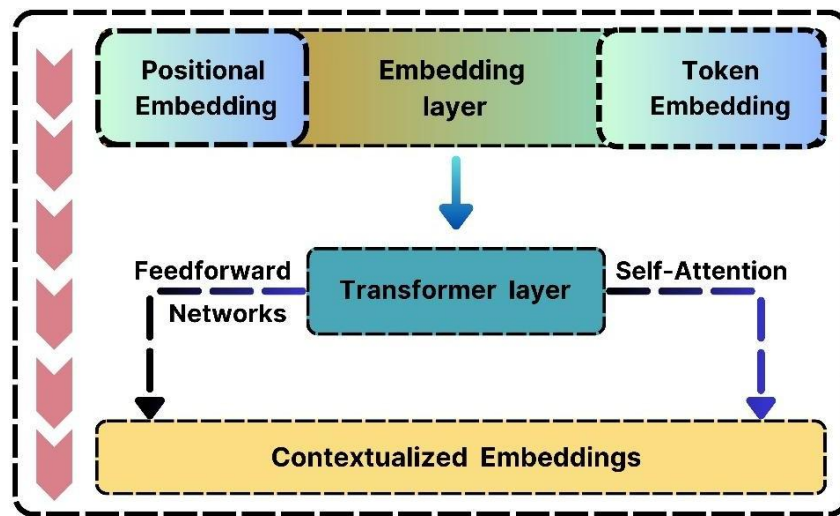


Figure 3: An overview of the CodeBERT archetchture.

4. Classifier Robustness and Stability Analysis

We carried out a thorough evaluation across various classification architectures over five independent test iterations, reporting both average and maximum performance metrics shown in Table 2, to evaluate the robustness and generalizability of our proposed CodeGraphNet embeddings beyond the single-layer neural network classifier reported in the main manuscript. The outcomes show a remarkable consistency in embedding quality where every classifier attains an $AUC > 0.90$, revealing an exceptionally high capacity for discrimination, confirming that the hierarchical graph fusion embeddings encapsulate linearly separable vulnerability patterns that even simplest classifiers can access. Additionally, stability is also confirmed by error metrics: mean squared error (MSE) constantly falls between 1.741 and 1.988 for all classifiers. The external validity of our findings is strengthened by this multi-classifier evaluation, which shows that the performance gains reported in the main manuscript are not artifacts of neural network-specific optimization dynamics but rather generalize across many learning paradigms.

Table 2: Performance Variability Analysis Across Classification Architectures (10-Fold Evaluation)

	Models	AUC	Acc	Pre	Rec	F1	MCC	Kappa	MSE	MAE
Average	Random Forest	0.932	0.762	0.773	0.762	0.766	0.715	0.714	1.779	0.588
	Gradient Boosting	0.928	0.746	0.758	0.746	0.751	0.696	0.696	1.959	0.636
	XGBoost	0.931	0.758	0.769	0.758	0.762	0.711	0.711	1.825	0.601
	SVM	0.916	0.764	0.781	0.764	0.769	0.719	0.717	1.789	0.586
	Logistic Regression	0.927	0.762	0.774	0.762	0.766	0.716	0.715	1.806	0.593
	MLP	0.903	0.749	0.759	0.749	0.752	0.701	0.699	1.832	0.611
	NN	0.901	0.778	0.779	0.767	0.771	0.721	0.719	1.741	0.578
Max	Random Forest	0.951	0.763	0.783	0.775	0.767	0.719	0.715	1.781	0.593
	Gradient Boosting	0.937	0.752	0.759	0.749	0.767	0.708	0.707	1.988	0.648
	XGBoost	0.947	0.762	0.773	0.764	0.771	0.735	0.732	1.899	0.624
	SVM	0.923	0.767	0.789	0.775	0.781	0.725	0.724	1.813	0.606
	Logistic Regression	0.952	0.775	0.779	0.768	0.773	0.747	0.731	1.927	0.605
	MLP	0.934	0.762	0.773	0.755	0.774	0.736	0.707	1.958	0.644
	NN	0.929	0.784	0.789	0.773	0.779	0.744	0.738	1.758	0.591

Moreover, we have further validated the results across 20 iterations for each of the classifiers on our independent test analysis and the results are depicted in Figure 4,5,6. In Figure 4, the consistent validation-test alignment across all classifiers with minimal divergence curves indicating insignificant overfitting. Notably, neural models including NN, GRU, LSTM and BERT exhibit validation losses oscillating between 0.85-1.2 while test losses remain stable at 0.2-0.3, highlighting the embeddings provide significant generalization, despite variations in validation. In contrast, tree-based models such as Decision Tree, Random Forest and DeepTree exhibit notable convergence patterns. At iteration 18, Random Forest displays a dramatic reduction in test loss from 1.3 to 1.0, whereas Decision Tree keeps straight trajectories at 0.88 and 1.35 for validation and test, respectively. Consecutively, Figure 5 reveals that Group 1 models including NN, GRU and LSTM achieve 0.775-0.786 of validation F1-scores with corresponding test scores of 0.765 to 0.771, exhibiting consistent 1-2% generalization gaps. Meanwhile Group 2 models including DeepTree, XGBoost, and SVM obtained closer convergence bands, with DeepTree displaying consistent test scores around 0.757 yet inconsistent validation performance ranging from 0.751 to 0.783. However, Group 3 models such as SGD, Decision Tree and Random Forest demonstrate SGD's substantial stability on test dataset alongside Random Forest. Figure 6 loss convergence rates, SGD from group 3 exhibits exponential decay from loss magnitude 6.0 to 1.0 where BERT from group 4 shifts between near 0 to 0.22, and Groups 2-3 maintain near zero losses throughout 20 iteration training. The consistent validation-test alignment across diverse classifiers substantiates that the proposed embeddings encode generalizable code semantics resilient to classifier-specific biases and training instabilities.

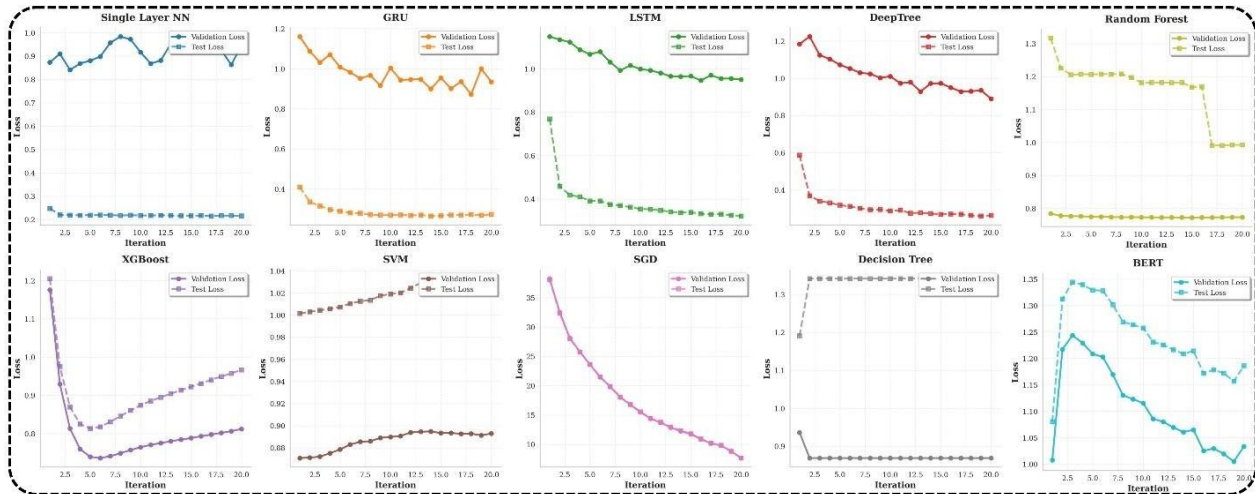


Figure 4: Validation and Test Loss Trajectories of Ten Classification Models on Proposed Embeddings Over 20 Iterations.

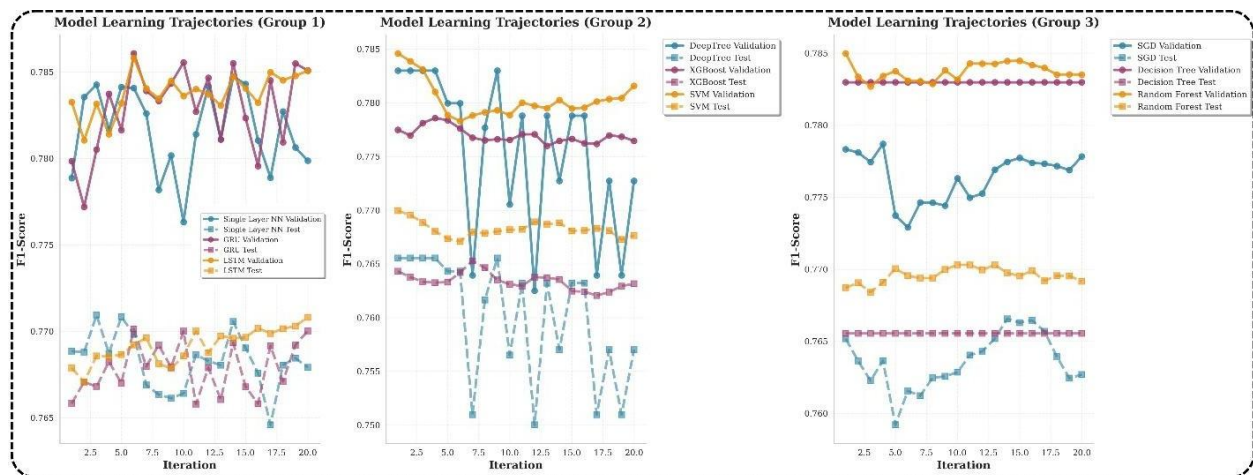


Figure 5: F1-Score Learning Trajectories Comparing Validation and Test Performance Across Model Architectures

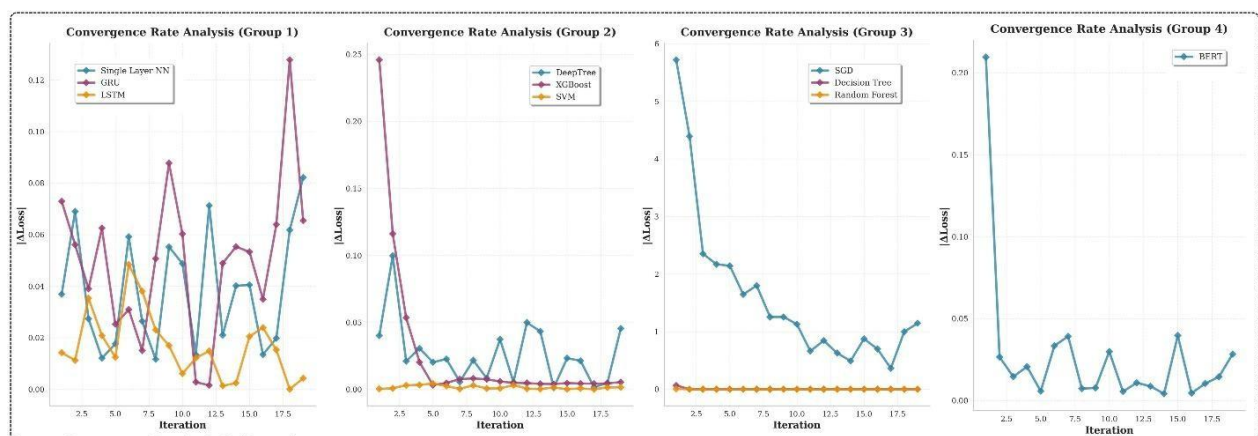
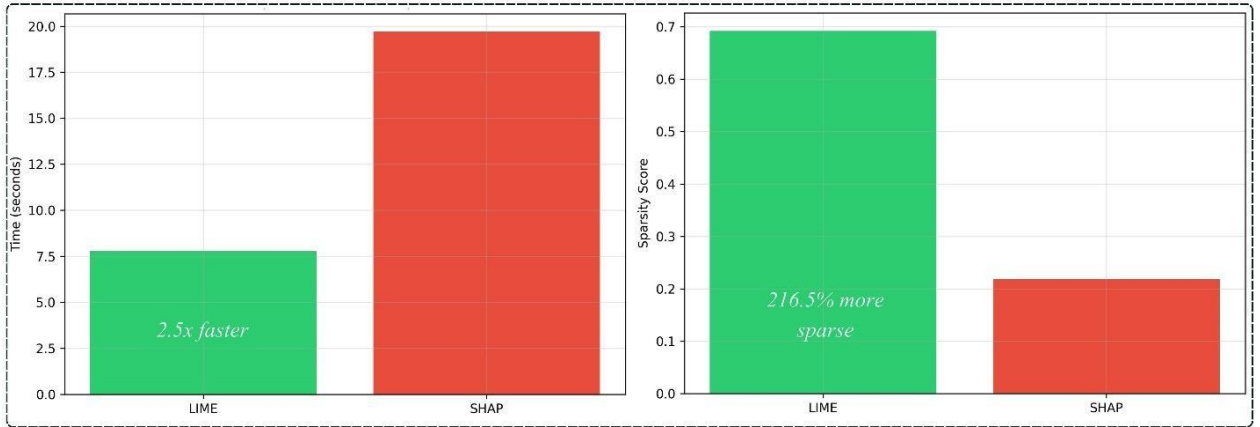


Figure 6: Loss Convergence Rate Analysis Across Four Model Groups During 20 Iteration

5. Explainability Method Validation

To empirically validate our selection of LIME over alternative explainability methods, we conducted a comprehensive quantitative comparison on 5 representative vulnerability samples from our Big_Vul test dataset. Each sample was analyzed using both LIME and SHAP (SHapley Additive exPlanations) with identical experimental settings. Figure 7 presents comparative analysis across two necessary dimensions of computational efficiency and explanation sparsity across 10 runs on 10 random code samples. Our empirical findings demonstrate that LIME achieves 2.5 $\times$ faster explanation generation (7.8 ± 0.4 seconds) compared to SHAP (19.7 ± 3.6 seconds), a substantial advantage for real-time vulnerability assessment workflows. More importantly, LIME exhibits 216.5% higher sparsity where LIME gains 0.69 and SHAP obtained 0.22, indicating substantial focus on corresponding output influential relevant tokens. The percentage of features with substantial relevance scores over the threshold (>0.01) is measured by this sparsity metric, where higher values reflect more concentrated, actionable explanations. Security professionals' demand for interpretable, targeted vulnerability localization is quickly fulfilled through LIME's focused token highlighting, whereas SHAP's diffuse importance distribution across numerous tokens reduces practical actionability for code-level remediation



tasks [7].

Figure 7: Computational efficiency and explanation sparsity comparison between LIME and SHAP methods. **Left panel:** Average computation time per explanation showing LIME's 2.5 $\times$ speedup advantage. **Right panel:** Sparsity scores demonstrating LIME's superior feature focus with 216.5% improvement over SHAP, enabling more actionable vulnerability localization for security practitioners.

References

- [1] Dey, R. and Salem, F.M., 2017, August. Gate-variants of gated recurrent unit (GRU) neural networks. In 2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS) (pp. 1597-1600). IEEE.
- [2] Dongare, A.D., Kharde, R.R. and Kachare, A.D., 2012. Introduction to artificial neural net- work. International Journal of Engineering and Innovative Technology (IJEIT), 2(1), pp.189- 194.
- [3] Jakkula, V., 2006. Tutorial on support vector machine (svm). School of EECS, Washington State University, 37(2.5), p.3.
- [4] Chen, T., 2015. Xgboost: extreme gradient boosting. R package version 0.4-2, 1(4).
- [5] Koroteev, M.V., 2021. BERT: a review of applications in natural language processing and understanding. arXiv preprint arXiv:2103.11943.
- [6] G. Jawahar, B. Sagot, and D. Seddah, “What Does BERT Learn about the Structure of Language?” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy, 2019, pp. 3651–3657, Association for Computational Linguistics.
- [7] Yutao Hu, Suyuan Wang, Wenke Li, Junru Peng, Yueming Wu, Deqing Zou, and Hai Jin. 2023. Interpreters for GNN-Based Vulnerability Detection: Are We There Yet? In Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 1407–1419. <https://doi.org/10.1145/3597926.3598145>